CivicSetu Architecture Audit

Current Modular Monolith and Target Microservices Migration

Audit date: November 9, 2026 **Repository baseline:** CivicSetu Django/React project **Validation:** 198 Django tests passed

Executive conclusion: CivicSetu is currently a modular monolith, not a microservices architecture. The nine-service Azure design is a planned target state; its migration phases have not started.

1. Current Architecture Assessment

Area	Current state	Assessment
Backend	One Django + Django REST Framework application	Modular monolith
Frontends	React web and Expo mobile clients call the same /api/v1/ API	Shared API surface
Database	Shared relational database with cross-app foreign keys	Strong coupling
Deployment	One backend deployment	Not independently deployable
Communication	Synchronous in-process Python calls	No broker or event bus
Files	Local backend media storage	Not horizontally scalable
Notifications	Created during request processing	No queue/retry system
Gateway/BFF	Django URL routing only	No implemented gateway or BFF
Infrastructure	No Dockerfiles, IaC, or service platform	Migration not started

2. Existing Domain Dependency Map

Accounts/User is referenced by Departments, Complaints, Notifications, Feedback and Audit logs. **Complaints** is referenced by Complaint History, Complaint Images, Notifications and Feedback. Authority dashboard services directly query complaint and notification models.

The most important coupling evidence is:

- Complaint directly references users and assigned authorities.
- Complaint History imports and references the Complaint model.
- Feedback directly references Complaint as a one-to-one relationship.
- Notifications directly reference both users and complaints.

3. Proposed Service Ownership

Target service	Current source modules
Identity	accounts and user/profile portions of users
Grievance	complaints and complaint_history
Department & Routing	departments and complaint scoping
Triage	Deterministic logic in complaints/services.py
Media	complaint_images and upload logic
Notification	notifications and notification logic in authority/services.py
Feedback	feedback
Reporting	Authority and department analytics
Audit	audit_logs
Web/Mobile BFF	Not implemented

4. Target Azure Architecture

The target design consists of nine independently deployable domain services and two BFFs: Identity, Grievance, Department & Routing, Triage, Media, Notification, Feedback, Reporting, Audit, Web BFF and Mobile BFF.

Planned infrastructure includes Azure Container Apps, PostgreSQL, Service Bus, Blob Storage, Front Door/WAF, Key Vault, Redis, Application Insights, Log Analytics and GitHub Actions with OIDC. These are planned components, not current deployments.

5. Database Decomposition Strategy

- Keep the shared database as the temporary system of record.
- Preserve stable public UUIDs and introduce versioned API contracts.
- Add an outbox table for reliable event publishing.
- Extract services incrementally, beginning with Notification.
- Move media to private Blob Storage with signed access URLs.
- Progress from shared tables to service-owned schemas and then separate databases where justified.
- Extract Identity last because it affects every client and service.

6. API and Event Contract Strategy

Preserve the existing /api/v1/ contract through an anti-corruption layer or BFF. Use versioned, idempotent events with event_id, event_type, occurred_at, version, source, entity_id, correlation_id and payload fields.

Recommended events: ComplaintCreated, ComplaintAssigned, ComplaintStatusChanged, ComplaintResolved, ComplaintRejected, EvidenceUploaded, FeedbackSubmitted, NotificationRequested and AuditEventRecorded.

7. Phased Migration Roadmap

Phase	Objective	Status
0	Baseline audit and contract inventory	Completed
1	Enforce module boundaries and remove direct cross-domain imports	Not started
2	Add contracts, UUID references and event/outbox foundation	Not started
3	Extract Notification service	Not started
4	Extract Media service and Blob Storage integration	Not started
5	Extract Feedback, Audit and Reporting	Not started
6	Extract Department/Routing and Triage	Not started
7	Extract Grievance service	Not started
8	Extract Identity service	Not started
9-10	Implement BFFs, Azure infrastructure, observability, backups and load testing	Not started

8. Key Risks

- Shared foreign keys prevent independent deployment.
- Authority services directly query complaint models, creating high coupling.
- Notifications occur inside request flows, complicating retries and failure recovery.
- Local media storage prevents reliable multi-instance deployment.
- The architecture documentation contains merge-conflict markers and should be cleaned.
- The deployment guide still describes a unified Django deployment.

9. Final Recommendation

Do not begin by creating nine separate Django projects. Start by enforcing internal domain boundaries and introducing contracts/events while preserving the working monolith. Then extract Notification as the first independently deployable service.

Generated from the verified repository baseline on November 09, 2026.